

Six-state toy model: thermodynamic inference tutorial

August 21, 2026

1 Six-state toy model: thermodynamic inference tutorial

This notebook walks through the complete package workflow for the six-state toy model:

1. load and inspect the cooled probability distributions;
2. plot the input data in the style used for the manuscript toy-model figure;
3. generate samples from the thermodynamically allowed region;
4. inspect the inferred hot-equilibrium distribution and sampling diagnostics; and
5. generate the four recovery figures.

The cooled probabilities are treated as exact, and all microstate entropies are fixed to zero. The hot- and cold-equilibrium CSV files are held-out references used only for plotting and evaluation; they are **not** supplied to the inference command.

This notebook retains 100,000 samples, matching the bundled publication-scale calculation, and writes them to a separate `tutorial_outputs/` directory.

1.1 Environment and paths

From this example directory, a source checkout can be installed once with:

```
python -m pip install -e ../../..
python -m pip install jupyterlab
```

The setup cell below also locates the local source tree automatically, so this notebook can be run before an editable installation. It contains no machine-specific paths.

```
[1]: from pathlib import Path
import json
import platform
import shlex
import shutil
import subprocess
import sys

import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scipy
from IPython.display import Image, Markdown, display
```

```

from matplotlib.ticker import AutoMinorLocator, LogLocator, NullFormatter

def locate_example_directory():
    """Find this example whether Jupyter starts here or above it."""
    start = Path.cwd().resolve()
    for base in (start, *start.parents):
        candidates = (
            base,
            base / "example_datasets" / "six_state_toy",
            base
            / "Thermodynamic_Inference_Package"
            / "example_datasets"
            / "six_state_toy",
        )
        for candidate in candidates:
            if (candidate / "cooled_distributions.csv").is_file():
                return candidate.resolve()
    raise FileNotFoundError(
        "Could not find example_datasets/six_state_toy. "
        "Start Jupyter from the package source checkout."
    )

EXAMPLE_DIR = locate_example_directory()
PACKAGE_ROOT = EXAMPLE_DIR.parents[1]

COOLED_CSV = EXAMPLE_DIR / "cooled_distributions.csv"
HOT_REFERENCE_CSV = EXAMPLE_DIR / "hot_equilibrium.csv"
COLD_REFERENCE_CSV = EXAMPLE_DIR / "cold_equilibrium.csv"
INFERENCE_DIR = EXAMPLE_DIR / "tutorial_outputs" / "inference"
FIGURE_DIR = EXAMPLE_DIR / "tutorial_outputs" / "figures"

INFERENCE_SCRIPT = PACKAGE_ROOT / "thermodynamic_inference.py"
PLOTING_SCRIPT = PACKAGE_ROOT / "plot_thermodynamic_recovery.py"
assert INFERENCE_SCRIPT.is_file() and PLOTING_SCRIPT.is_file()

print(f"Example directory: {EXAMPLE_DIR}")
print(f"Python {platform.python_version()}")
print(
    f"NumPy {np.__version__}; SciPy {scipy.__version__}; "
    f"pandas {pd.__version__}; Matplotlib {matplotlib.__version__}"
)

```

Example directory: <package-root>/example_datasets/six_state_toy
 Python 3.12.7
 NumPy 1.26.4; SciPy 1.13.1; pandas 2.2.2; Matplotlib 3.9.2

1.2 Load and inspect the input data

The cooled input is a long-form CSV with one row per cooling time and state. Its confidence-limit columns are blank in this example, so the probabilities are treated as exact.

```
[2]: cooled = pd.read_csv(COOLED_CSV)
hot_reference = pd.read_csv(HOT_REFERENCE_CSV)
cold_reference = pd.read_csv(COLD_REFERENCE_CSV)

display(cooled.head(12))

state_order = cooled["state"].drop_duplicates().tolist()
probability_table = (
    cooled.pivot(index="cooling_time", columns="state", values="probability")
    .loc[:, state_order]
    .sort_index()
)

normalization = probability_table.sum(axis=1).rename("probability_sum")
assert np.allclose(normalization.to_numpy(), 1.0)
assert cooled[["ci_lower_95", "ci_upper_95"]].isna().all().all()

print(
    f"Loaded {len(probability_table)} cooled distributions over "
    f"{len(state_order)} states."
)
display(probability_table)
display(normalization.to_frame())
```

	cooling_time	state	probability	ci_lower_95	ci_upper_95
0	1000	state_1	0.0665	NaN	NaN
1	1000	state_2	0.2219	NaN	NaN
2	1000	state_3	0.1465	NaN	NaN
3	1000	state_4	0.1021	NaN	NaN
4	1000	state_5	0.3013	NaN	NaN
5	1000	state_6	0.1617	NaN	NaN
6	5000	state_1	0.0282	NaN	NaN
7	5000	state_2	0.2623	NaN	NaN
8	5000	state_3	0.1420	NaN	NaN
9	5000	state_4	0.0794	NaN	NaN
10	5000	state_5	0.3394	NaN	NaN
11	5000	state_6	0.1487	NaN	NaN

Loaded 10 cooled distributions over 6 states.

state	state_1	state_2	state_3	state_4	state_5	state_6
cooling_time						
1000	0.0665	0.2219	0.1465	0.1021	0.3013	0.1617
5000	0.0282	0.2623	0.1420	0.0794	0.3394	0.1487
10000	0.0206	0.2709	0.1454	0.0732	0.3538	0.1361

50000	0.0051	0.2649	0.1514	0.0555	0.4120	0.1111
100000	0.0040	0.2631	0.1512	0.0414	0.4445	0.0958
500000	0.0028	0.2319	0.1469	0.0331	0.5137	0.0716
1000000	0.0019	0.2106	0.1517	0.0236	0.5534	0.0588
5000000	0.0017	0.1914	0.1582	0.0182	0.5917	0.0388
10000000	0.0014	0.1759	0.1541	0.0132	0.6221	0.0333
50000000	0.0019	0.1481	0.1532	0.0115	0.6617	0.0236

	probability_sum
cooling_time	
1000	1.0
5000	1.0
10000	1.0
50000	1.0
100000	1.0
500000	1.0
1000000	1.0
5000000	1.0
10000000	1.0
50000000	1.0

1.3 Plot the input distributions

Solid lines show the cooled distributions. The same-color dotted lines show the supplied hot-equilibrium reference, matching the input-data panel of the manuscript toy-model figure. Those reference values are shown for orientation only and remain excluded from inference.

```
[3]: def format_input_axes(ax):
    """Match the boxed, inward-tick style of the manuscript panel."""
    ax.xaxis.set_minor_locator(LogLocator(subs="auto"))
    ax.xaxis.set_minor_formatter(NullFormatter())
    ax.yaxis.set_minor_locator(AutoMinorLocator(2))
    ax.tick_params(
        which="major", width=0.72, length=4.5, direction="in",
        top=True, right=True, labelsiz=8,
    )
    ax.tick_params(
        which="minor", width=0.54, length=2.25, direction="in",
        top=True, right=True,
    )
    for spine in ax.spines.values():
        spine.set_color("black")
        spine.set_linewidth(0.72)

markers = ["o", "s", "^", "D", "v", "<"]
colors = [
    "tab:blue", "tab:orange", "tab:green",
```

```

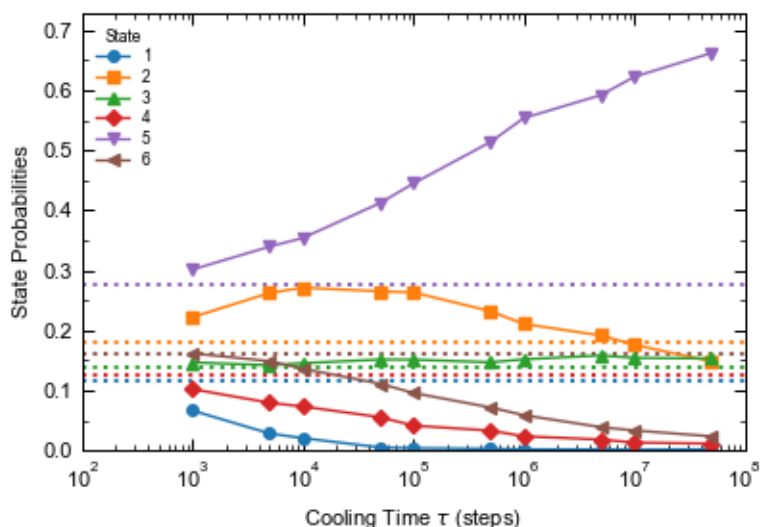
        "tab:red", "tab:purple", "tab:brown",
    ]
    hot_by_state = hot_reference.set_index("state").loc[state_order, "probability"]

    with plt.rc_context(
        {
            "font.family": "sans-serif",
            "font.sans-serif": ["Arial", "Helvetica", "DejaVu Sans"],
            "text.usetex": False,
            "axes.labelsize": 8,
            "legend.fontsize": 6,
        }
    ):
        fig, ax = plt.subplots(figsize=(4.2, 3.0))
        for index, state in enumerate(state_order):
            ax.plot(
                probability_table.index,
                probability_table[state],
                color=colors[index],
                marker=markers[index],
                linewidth=1,
                markersize=4,
                label=state.removeprefix("state_"),
                zorder=2,
            )
            ax.hlines(
                hot_by_state[state],
                1e2,
                1e8,
                color=colors[index],
                linestyle=":",
                linewidth=1.5,
                zorder=1,
            )

        ax.set_xscale("log")
        ax.set_xlim(1e2, 1e8)
        maximum_probability = max(
            probability_table.to_numpy().max(), hot_by_state.to_numpy().max()
        )
        ax.set_ylim(0, 1.1 * maximum_probability)
        ax.set_xlabel(r"Cooling Time  $\tau$  (steps)")
        ax.set_ylabel("State Probabilities")
        ax.legend(
            loc="upper left", frameon=False, labelspacing=0.2, title="State",
            title_fontsize=6,
        )

```

```
format_input_axes(ax)
fig.tight_layout()
plt.show()
```



1.4 Run the thermodynamic inference

The package is primarily a command-line tool. Calling it through `subprocess` from the notebook uses the same interface as a terminal while preserving the active Python environment.

For this exact six-state example, the automatic method is scrambled-Sobol rejection sampling in the probability simplex. `--no-microstate-entropies` fixes all state entropies to zero. The smallest numerical cooling time identifies the fastest cooled run; the numerical spacing between times is not otherwise used.

After an installed-package setup, the equivalent terminal command begins with `thermodynamic-inference` rather than `python thermodynamic_inference.py`.

```
[4]: N_SAMPLES = 100_000 # Publication-scale setting used by the bundled result.
SEED = 20_260_814

inference_command = [
    sys.executable,
    str(INFERENCE_SCRIPT),
    "--cooled", str(COOLED_CSV),
    "--hot-temperature", "500",
    "--cold-temperature", "100",
    "--no-microstate-entropies",
    "--n-samples", str(N_SAMPLES),
    "--workers", "8",
    "--seed", str(SEED),
```

```

    "--output-dir", str(INFERENCE_DIR),
    "--overwrite",
]

print("$", shlex.join(inference_command))
completed = subprocess.run(
    inference_command,
    cwd=EXAMPLE_DIR,
    check=True,
    text=True,
    capture_output=True,
)
print(completed.stdout)
if completed.stderr:
    print(completed.stderr, file=sys.stderr)

```

```

$ python <package-root>/thermodynamic_inference.py --cooled <package-
root>/example_datasets/six_state_toy/cooled_distributions.csv --hot-temperature
500 --cold-temperature 100 --no-microstate-entropies --n-samples 100000
--workers 8 --seed 20260814 --output-dir <package-
root>/example_datasets/six_state_toy/tutorial_outputs/inference --overwrite

```

```

Loaded 10 cooling distributions over 6 states (exact input).
Sampling the exact-data region by scrambled-simplex rejection...
  exact simplex: 459/100,000 accepted from 2,097,152 proposals
  ... 224 intermediate progress updates omitted from the rendered PDF ...
  exact simplex: 100,403/100,000 accepted from 473,956,352 proposals
Saved 100,000 samples to <package-
root>/example_datasets/six_state_toy/tutorial_outputs/inference
Point estimator: center

```

1.5 Inspect the generated outputs

The compact NPZ archive contains every sampled probability distribution. The CSV files provide convenient summaries, diagnostics, and the reported point estimate. For the default **center** estimator, the estimate is the componentwise sample mean and its uncertainty column is the population standard deviation of the generated allowed samples.

```

[5]: output_files = sorted(path.name for path in INFERENCE_DIR.iterdir())
print("Generated files:")
for name in output_files:
    print(f"  {name}")

estimate = pd.read_csv(INFERENCE_DIR / "hot_equilibrium_estimate.csv")
region_summary = pd.read_csv(INFERENCE_DIR / "allowed_region_summary.csv")
diagnostics = pd.read_csv(INFERENCE_DIR / "sampling_diagnostics.csv")
metadata = json.loads((INFERENCE_DIR / "run_metadata.json").read_text())

```

```

with np.load(INFERENCE_DIR / "allowed_samples.npz", allow_pickle=False) as archive:
    allowed_probabilities = archive["probabilities"].copy()
    allowed_entropies = archive["entropies"].copy()
    implied_cold_probabilities = archive["implied_cold_probabilities"].copy()
    saved_states = archive["state_labels"].astype(str).tolist()

assert allowed_probabilities.shape == (N_SAMPLES, len(state_order))
assert np.allclose(allowed_probabilities.sum(axis=1), 1.0)
assert np.all(allowed_probabilities >= 0)
assert np.allclose(allowed_entropies, 0.0)
assert saved_states == state_order

estimate_by_state = estimate.set_index("state").loc[state_order]
np.testing.assert_allclose(
    estimate_by_state["estimate"], allowed_probabilities.mean(axis=0)
)
np.testing.assert_allclose(
    estimate_by_state["standard_deviation"],
    allowed_probabilities.std(axis=0, ddof=0),
)

assert implied_cold_probabilities.shape == allowed_probabilities.shape
assert np.allclose(implied_cold_probabilities.sum(axis=1), 1.0)

display(estimate_by_state[["estimate", "standard_deviation"]])
display(
    region_summary[
        ["state", "probability_mean", "probability_std",
         "probability_min", "probability_max"]
    ]
)

```

Generated files:

```

allowed_region_summary.csv
allowed_samples.npz
hot_equilibrium_estimate.csv
run_metadata.json
sampling_diagnostics.csv

```

	estimate	standard_deviation
state		
state_1	0.108206	0.024122
state_2	0.176709	0.021133
state_3	0.160670	0.025805
state_4	0.127366	0.024922
state_5	0.275467	0.018326

state_6	0.151581	0.023160	
---------	----------	----------	--

	state	probability_mean	probability_std	probability_min \
0	state_1	0.108206	0.024122	0.051186
1	state_2	0.176709	0.021133	0.127491
2	state_3	0.160670	0.025805	0.098468
3	state_4	0.127366	0.024922	0.069577
4	state_5	0.275467	0.018326	0.225546
5	state_6	0.151581	0.023160	0.098205

	probability_max
0	0.182786
1	0.242763
2	0.235401
3	0.202749
4	0.325948
5	0.224177

```
[6]: run_summary = pd.Series(
    {
        "method": metadata["method"],
        "retained samples": metadata["n_samples"],
        "raw proposals": metadata["sampling"]["total_proposals"],
        "raw acceptance fraction":_
    metadata["sampling"]["raw_acceptance_fraction"],
        "elapsed seconds": metadata["sampling"]["elapsed_seconds"],
        "minimum saved constraint margin":_
    metadata["validation"]["minimum_saved_manuscript_margin"],
        "maximum normalization error":_
    metadata["validation"]["maximum_probability_normalization_error"],
    },
    name="value",
)
display(run_summary.to_frame())
display(diagnostics.head())
```

	value
method	exact_simplex_qmc_rejection
retained samples	100000
raw proposals	473956352
raw acceptance fraction	0.000212
elapsed seconds	80.657112
minimum saved constraint margin	0.000001
maximum normalization error	0.0

	batch_id	proposal_count	entropy_domain_count	accepted_count \
0	0	262144	262144	56
1	1	262144	262144	54
2	2	262144	262144	51

3	3	262144	262144	63
4	4	262144	262144	52

	accepted_fraction	minimum_accepted_margin
0	0.000214	0.000178
1	0.000206	0.000276
2	0.000195	0.000088
3	0.000240	0.000629
4	0.000198	0.000614

1.6 Compare the estimate with the held-out equilibrium

This comparison evaluates recovery after inference is complete. It does not feed either equilibrium reference back into the algorithm. The KL divergence uses the inferred or cooled distribution as its first argument and the supplied hot equilibrium as its reference.

```
[7]: hot_by_state = hot_reference.set_index("state").loc[state_order, "probability"]
inferred = estimate_by_state["estimate"].to_numpy()
hot = hot_by_state.to_numpy()
fastest_cooled = probability_table.iloc[0].to_numpy()

def kl_divergence(distribution, reference):
    return np.sum(distribution * np.log(distribution / reference))

comparison = pd.DataFrame(
    {
        "state": state_order,
        "fastest_cooled": fastest_cooled,
        "inferred": inferred,
        "held_out_hot_equilibrium": hot,
    }
)
metrics = pd.DataFrame(
    {
        "KL divergence to hot equilibrium": [
            kl_divergence(fastest_cooled, hot),
            kl_divergence(inferred, hot),
        ],
        "total variation distance": [
            0.5 * np.abs(fastest_cooled - hot).sum(),
            0.5 * np.abs(inferred - hot).sum(),
        ],
    },
    index=["fastest cooled", "inferred"],
)
```

```
display(comparison)
display(metrics)
```

	state	fastest_cooled	inferred	held_out_hot_equilibrium
0	state_1	0.0665	0.108206	0.1154
1	state_2	0.2219	0.176709	0.1807
2	state_3	0.1465	0.160670	0.1383
3	state_4	0.1021	0.127366	0.1271
4	state_5	0.3013	0.275467	0.2776
5	state_6	0.1617	0.151581	0.1609

	KL divergence to hot equilibrium	total variation distance
fastest cooled	0.020483	0.073900
inferred	0.002276	0.022637

1.7 Generate the recovery figures

The second package command reads the original cooled CSV and the inference directory. The equilibrium references are supplied here because they are plotting and recovery-evaluation inputs. We explicitly request the state-2/state-5 slice used for this example.

Package figures require a LaTeX installation; PNG output additionally requires `dvipng`. If either executable is unavailable, this cell reports the missing dependency and leaves the inference results intact. After installation, the equivalent terminal command begins with `thermodynamic-recovery-plot`.

```
[8]: missing_plot_tools = [
    command for command in ("latex", "dvipng") if shutil.which(command) is None
]

if missing_plot_tools:
    print(
        "Skipping package recovery figures; missing system commands: "
        + ", ".join(missing_plot_tools)
    )
else:
    plotting_command = [
        sys.executable,
        str(PLOTTING_SCRIPT),
        "--cooled", str(COOLED_CSV),
        "--inference-dir", str(INFERENCE_DIR),
        "--hot-equilibrium", str(HOT_REFERENCE_CSV),
        "--cold-equilibrium", str(COLD_REFERENCE_CSV),
        "--slice-states", "state_2", "state_5",
        "--time-unit", "simulation steps",
        "--format", "png",
        "--dpi", "150",
        "--seed", "20260820",
        "--output-dir", str(FIGURE_DIR),
```

```

        "--overwrite",
    ]
    print("$", shlex.join(plotting_command))
    completed = subprocess.run(
        plotting_command,
        cwd=EXAMPLE_DIR,
        check=True,
        text=True,
        capture_output=True,
    )
    print(completed.stdout)
    if completed.stderr:
        print(completed.stderr, file=sys.stderr)

```

```

$ python <package-root>/plot_thermodynamic_recovery.py --cooled <package-
root>/example_datasets/six_state_toy/cooled_distributions.csv --inference-dir
<package-root>/example_datasets/six_state_toy/tutorial_outputs/inference --hot-
equilibrium <package-root>/example_datasets/six_state_toy/hot_equilibrium.csv
--cold-equilibrium <package-
root>/example_datasets/six_state_toy/cold_equilibrium.csv --slice-states state_2
state_5 --time-unit 'simulation steps' --format png --dpi 150 --seed 20260820
--output-dir <package-
root>/example_datasets/six_state_toy/tutorial_outputs/figures --overwrite

```

Figure 1 slice states: 'state_2' and 'state_5'

Saved <package-root>/example_datasets/six_state_toy/tutorial_outputs/figures/figure_1_state_space.png

Saved <package-root>/example_datasets/six_state_toy/tutorial_outputs/figures/figure_2_state_probabilities.png

Saved <package-root>/example_datasets/six_state_toy/tutorial_outputs/figures/figure_3_entropy_production.png and <package-root>/example_datasets/six_state_toy/tutorial_outputs/figures/figure_3_entropy_production_data.csv

Saved <package-root>/example_datasets/six_state_toy/tutorial_outputs/figures/figure_4_kl_divergence.png and <package-root>/example_datasets/six_state_toy/tutorial_outputs/figures/figure_4_kl_divergence_data.csv

```

[9]: figure_paths = (
    sorted(Figure_DIR.glob("figure_[1-4]_*.png"))
    if Figure_DIR.is_dir()
    else []
)

if not figure_paths:
    print("No tutorial figures were generated.")
else:
    for path in figure_paths:
        display(Markdown(f"**{path.stem.replace('_', ' ').title()}**"))

```

```
display(Image(filename=str(path), width=650))
```

Figure 1: State-space recovery

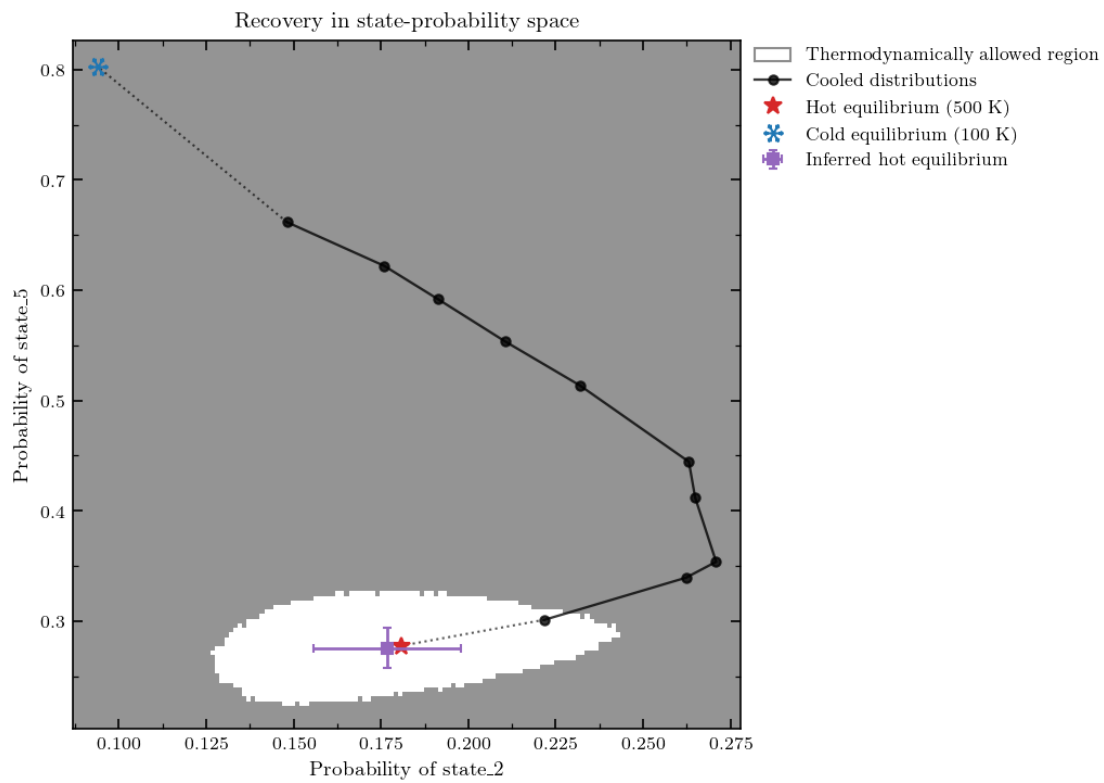


Figure 2: State-probability recovery

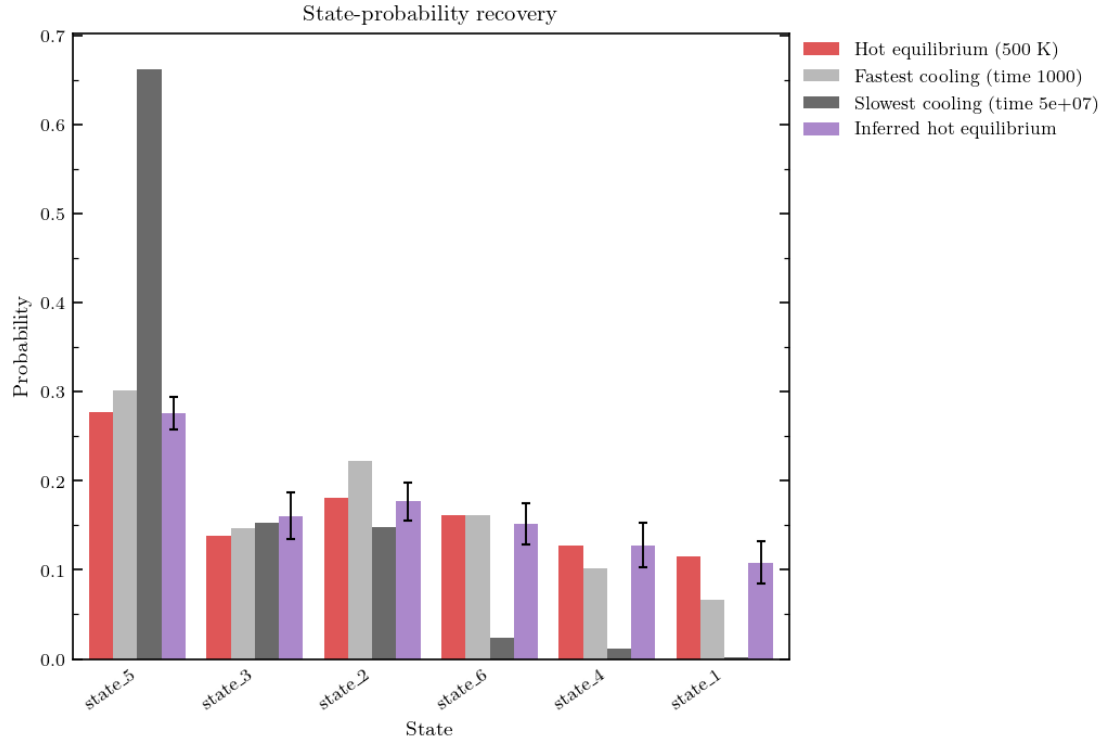


Figure 3: Entropy production

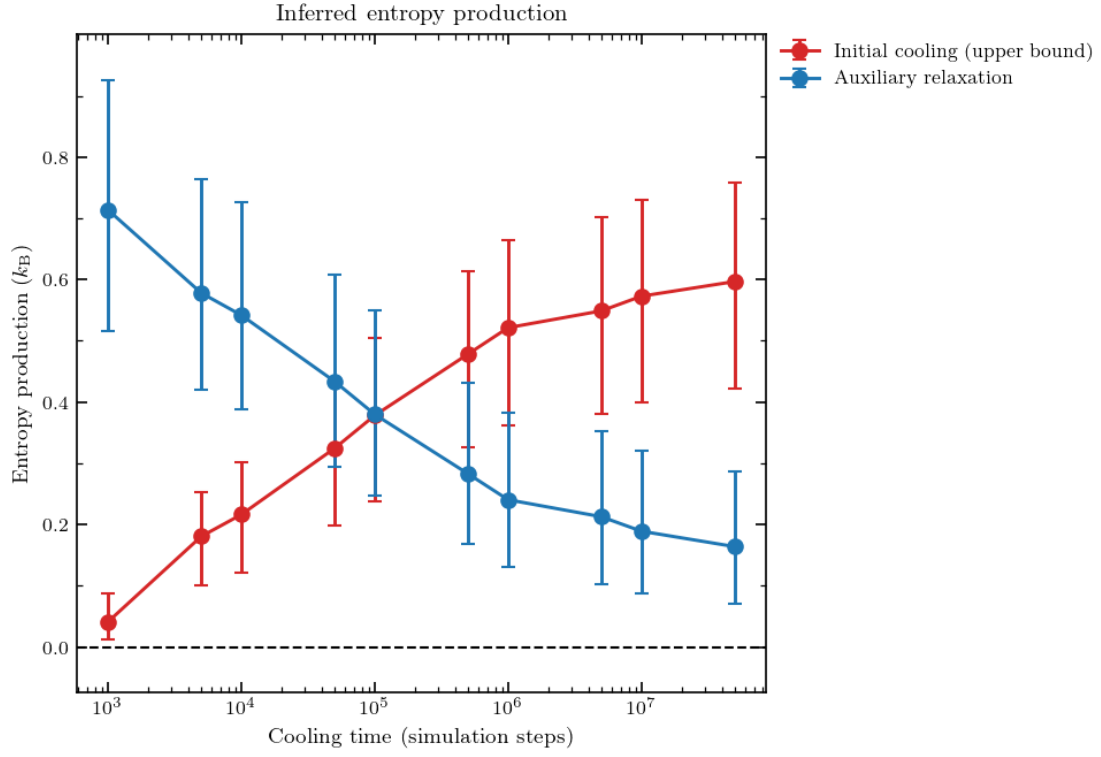
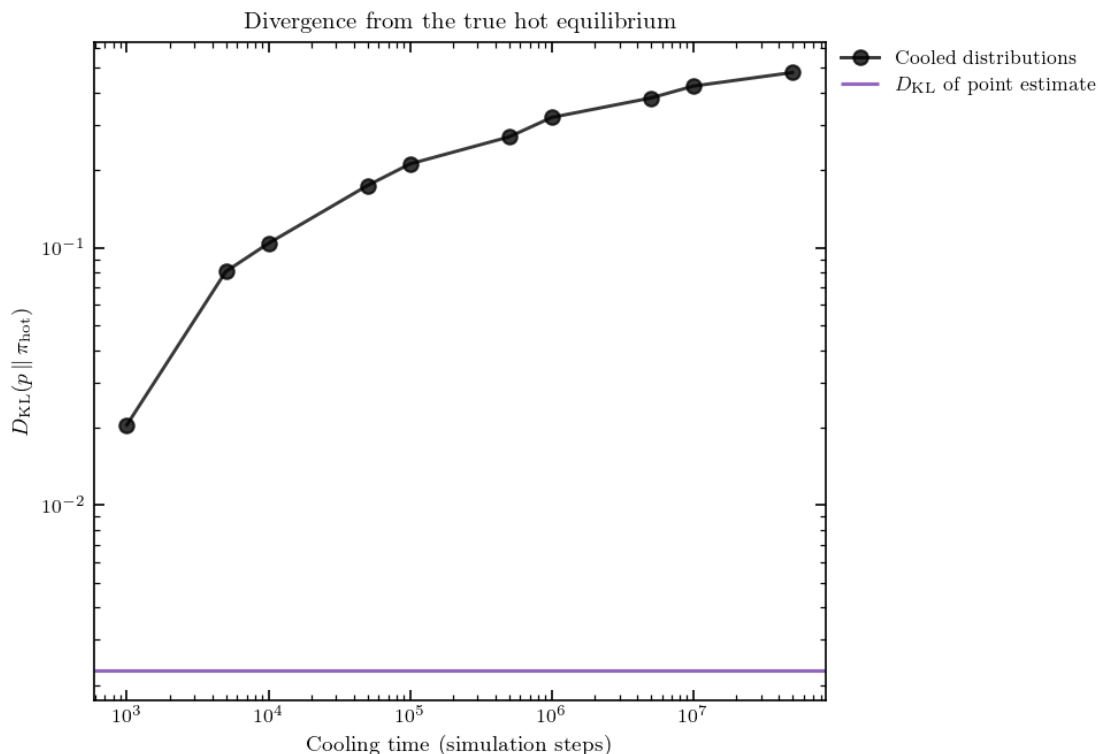


Figure 4: KL divergence



1.8 Full-scale and exploratory runs

This notebook already uses the publication-scale setting of 100,000 retained samples and eight workers. For a faster exploratory run, reduce `N_SAMPLES` and optionally use one worker. The bundled `recovery_outputs/` and `figures/` directories contain the saved reference result, so keep experimental runs in a separate output directory unless you intentionally want to replace those artifacts.

For datasets with confidence intervals or inferred microstate entropies, see the main package README before changing the corresponding command-line options; those cases have a different interpretation from this exact, zero-entropy example.